Unit 1 – Using Objects and Methods
Lesson 4: Casting and Range of Variables



Maximum and Minimum Integer Values

- Integer values in Java represented as an `int` are stored using a finite amount (4 bytes, or 32 bits) of memory
- Therefore, an `int` value must be in the range from `Integer.MIN_VALUE` to `Integer.MAX_VALUE`
- These constants are part of the `Integer` class, which belongs to the `java.lang` package. This package is automatically imported into every Java program, so you can use `Integer.MIN_VALUE` and `Integer.MAX_VALUE` without needing to write an import statement.
- If an expression would evaluate to an `int` value outside of the allowed range, an integer overflow occurs

`Integer.MIN_VALUE`	Smallest integer allowed: –2,147,483,648 (or -2^{31})	
`Integer.MAX_VALUE`	Largest integer allowed: 2,147,483,647 (or $2^{31} - 1$)	

Casting

- The casting operators `(int)` and `(double)` can be used to convert from a `double` value to an `int` value (or vice versa)
- When mixing expressions, we don't have to use the casting operator, but it can help make our code clearer:

Casting without the operator	**Casting with the operator**

What is printed?
 	
`int x = 9;`
`int y = 2;`
`double z = 1.0 * x / y;`
`System.out.println(z);`
	What is printed?
	
	`int x = 9;`
	`int y = 2;`
	`double z = (double) x / y;`
	`System.out.println(z);`

- Casting changes a variable from one data type to another, *temporarily*
- In our example above, `x` does not "become" a double in memory, it is just upgraded for the expression
- The example above represents a widening conversion – where we move the data type into a "bigger box" temporarily
- A narrowing conversion is when you move the data type into a "smaller box"

Widening Conversion

- `double z = 1.0 * x / y;` is a widening conversion and something that Java will do automatically in mixed expressions
- `double z = (double) x / y;` is also a widening conversion, but an explicit one

Code	Value	Reason
`double x = 7/2;`		Java still performs integer division and then stores it in a double.
`double y = (double)7/2;`		The cast happens first on the 7. 7 will get promoted to a 7.0 and then we have a mixed expression 7.0/2, which Java will then promote the 2 to a 2.0 and double division is performed.
`double z = (double) (7/2);`		The parentheses are evaluated first, and integer division is performed. Then the cast happens on the result.
`int a = (double)7/2;`		The 7 is promoted to a 7.0 and then the mixed expression 7.0/2 is evaluated. However, this results in an answer that is a double, which will not fit in an int value. This will result in a compile time error called "possible lossy conversion"

Narrowing Conversion

- If we don't explicitly cast when moving from a larger data type to a smaller one, we'll get a _____ called "possible lossy conversion"
- If we want to do a narrowing conversion, we must _____
- If we change from a double to an int, we lose information (the decimal part), by _____, NOT ROUNDING. Truncation "chops off" the decimal in the double value.

Code	Value	Reason
`int x = 3.5;`		This statement does not use an explicit cast and the compiler error "possible lossy conversion" will be displayed.
`int y = (int) 3.55;`		Explicitly casting down will "cut off" the decimal to store in an integer value
`int z = (int) 3.99;`		No rounding will take place when explicitly casting down, only truncation.

Rounding Decimals

- How do we round decimals if Java will automatically truncate?
- There are a few methods we might see that can help, but an easy way to do it when casting involves a number trick:

Code	Why Does It Work?
`double value = 3.7;` `int myInt = (int)(value + 0.5);`	• `value` will become 4.2 first in paratheses and then is truncated with the explicit cast down • `myInt` will be 4, which properly rounds 3.7
`double value = 3.3;` `int myInt = (int)(value + 0.5);`	• `value` will become 3.8 first in paratheses and then is truncated with the explicit cast down • `myInt` will be 3, which properly rounds 3.3